

Vulkan Notes

Uniform Buffer Objects & Descripteurs

Adam HAMOUCH

June 2026

1. Pourquoi les Uniform Buffer Objects

Un UBO est un buffer GPU dont le contenu est accessible en lecture seule et identique pour toutes les invocations d'un shader pendant un draw call. C'est la solution pour passer des données **globales à la frame** aux shaders sans les dupliquer dans chaque vertex.

Sans UBO, les seules options seraient de modifier le vertex buffer chaque frame (coûteux, implique un staging buffer) ou d'utiliser des push constants (limitées à 128 octets). Les UBOs permettent de passer des matrices, des paramètres de lumière, du temps, etc., sans toucher à la géométrie.

Ce qui va dans un UBO : tout ce qui est uniforme pour un draw — matrices model/view/proj, position caméra, temps, paramètres de lumière, couleurs globales.

Ce qui n'y va pas : données par vertex (vertex buffer), données qui changent très souvent par objet (push constants), textures (sampler séparé).

2. Descriptor Set Layout

Le **descriptor set layout** décrit la structure des ressources attendues par le shader — les types et stages, pas les données réelles. C'est un contrat entre le pipeline et les shaders.

```
VkDescriptorSetLayoutBinding uboBinding{};
uboBinding.binding           = 0;                               // binding=0 dans le shader
uboBinding.descriptorType    = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
uboBinding.descriptorCount    = 1;
uboBinding.stageFlags        = VK_SHADER_STAGE_VERTEX_BIT; // visible par le vertex
                           shader

VkDescriptorSetLayoutCreateInfo layoutInfo{};
layoutInfo.bindingCount = 1;
layoutInfo.pBindings    = &uboBinding;
vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr, &descriptorSetLayout);
```

Ce layout est ensuite passé au pipeline layout lors de la creation de la pipeline :

```
pipelineLayoutInfo.setLayoutCount = 1;
pipelineLayoutInfo.pSetLayouts    = &descriptorSetLayout;
```

3. Uniform Buffer & Mise a jour

Un UBO est alloué en `HOST_VISIBLE | HOST_COHERENT` — pas de staging buffer car il est mis à jour à chaque frame. Un buffer par image de la swap chain pour éviter que le CPU écrase les données pendant que le GPU lit la frame précédente.

```
// Creation : un buffer par image SC
uniformBuffers.resize(swapChainImages.size());
for (size_t i = 0; i < swapChainImages.size(); i++)
    createBuffer(sizeof(UBO),
        VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT,
        VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
        uniformBuffers[i], uniformBuffersMemory[i]);

// Mise a jour chaque frame
void updateUniformBuffer(uint32_t currentImage) {
    UniformBufferObject ubo{};
    ubo.model = ...;
    ubo.view = camera->GetViewMatrix();
    ubo.proj = camera->GetProjectionMatrix();

    void* data;
    vkMapMemory(device, uniformBuffersMemory[currentImage], 0, sizeof(ubo), 0, &data
    );
    memcpy(data, &ubo, sizeof(ubo));
    vkUnmapMemory(device, uniformBuffersMemory[currentImage]);
}
```

4. Descriptor Pool & Descriptor Sets

Les descriptor sets ne peuvent pas être créés directement — ils sont alloués depuis une pool, comme les command buffers. La pool réserve un bloc mémoire pour un nombre fixe de sets et de descripteurs.

```
// Pool
VkDescriptorPoolSize poolSize{};
poolSize.type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
poolSize.descriptorCount = static_cast<uint32_t>(swapChainImages.size());

VkDescriptorPoolCreateInfo poolInfo{};
poolInfo.poolSizeCount = 1;
poolInfo.pPoolSizes = &poolSize;
poolInfo.maxSets = static_cast<uint32_t>(swapChainImages.size());
vkCreateDescriptorPool(device, &poolInfo, nullptr, &descriptorPool);

// Allocation des sets
std::vector<VkDescriptorSetLayout> layouts(swapChainImages.size(),
    descriptorSetLayout);
VkDescriptorSetAllocateInfo allocInfo{};
allocInfo.descriptorPool = descriptorPool;
allocInfo.descriptorSetCount = static_cast<uint32_t>(swapChainImages.size());
allocInfo.pSetLayouts = layouts.data();
vkAllocateDescriptorSets(device, &allocInfo, descriptorSets.data());

// Ecriture : faire pointer chaque set vers son buffer
for (size_t i = 0; i < swapChainImages.size(); i++) {
    VkDescriptorBufferInfo bufferInfo{};
    bufferInfo.buffer = uniformBuffers[i];
    bufferInfo.offset = 0;
    bufferInfo.range = sizeof(UniformBufferObject);

    VkWriteDescriptorSet write{};
    write.dstSet = descriptorSets[i];
    write.dstBinding = 0;
    write.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
    write.descriptorCount = 1;
    write.pBufferInfo = &bufferInfo;
    vkUpdateDescriptorSets(device, 1, &write, 0, nullptr);
}
```

Binding dans recordCommandBuffer : le descriptor set doit être bind avant le draw, avec le bon index correspondant à l'image courante.

```
vkCmdBindDescriptorSets(commandBuffer,
    VK_PIPELINE_BIND_POINT_GRAPHICS,
    pipelineLayout, 0, 1,
    &descriptorSets[imageIndex], 0, nullptr);
vkCmdDrawIndexed(...);
```

5. Alignement memoire (std140)

Vulkan exige un alignement precis des membres d'un UBO en memoire (layout std140). Si un membre n'est pas au bon offset, le shader lit des donnees corrompues — ecran noir sans erreur de validation.

Regles d'alignement :

Type	Alignement	Offset requis
float	4 octets	multiple de 4
vec2	8 octets	multiple de 8
vec3	16 octets	multiple de 16
vec4	16 octets	multiple de 16
mat4	16 octets	multiple de 16

Trois `mat4` consecutives sont naturellement alignees (offsets 0, 64, 128 — tous multiples de 16). Mais ajouter un `vec2` avant decale tout de 8 octets et casse l'alignement.

Solution : toujours etre explicite avec `alignas(16)` :

```
struct UniformBufferObject {
    alignas(16) Mat4 model;
    alignas(16) Mat4 view;
    alignas(16) Mat4 proj;
};
```

Vulkan Y-flip : Vulkan a le clip space Y inverse par rapport a OpenGL. Sans correction, la geometrie est rendue a l'envers et le back-face culling la rejette → ecran noir. Corriger en inversant `proj[1][1] *= -1` apres avoir calcule la matrice de projection.

Plusieurs sets de descripteurs : on peut binder plusieurs sets simultanement. Convention standard : `set=0` pour les donnees par frame (view, proj, temps), `set=1` pour les donnees par objet (model matrix). Cela evite de rebinder les donnees partagees a chaque draw call.

```
layout(set = 0, binding = 0) uniform FrameUBO { mat4 view; mat4 proj; } frame;
layout(set = 1, binding = 0) uniform ObjectUBO { mat4 model; } obj;
```